

RAG: Theory & Architecture

A Deep Dive into Retrieval-Augmented Generation Workflow

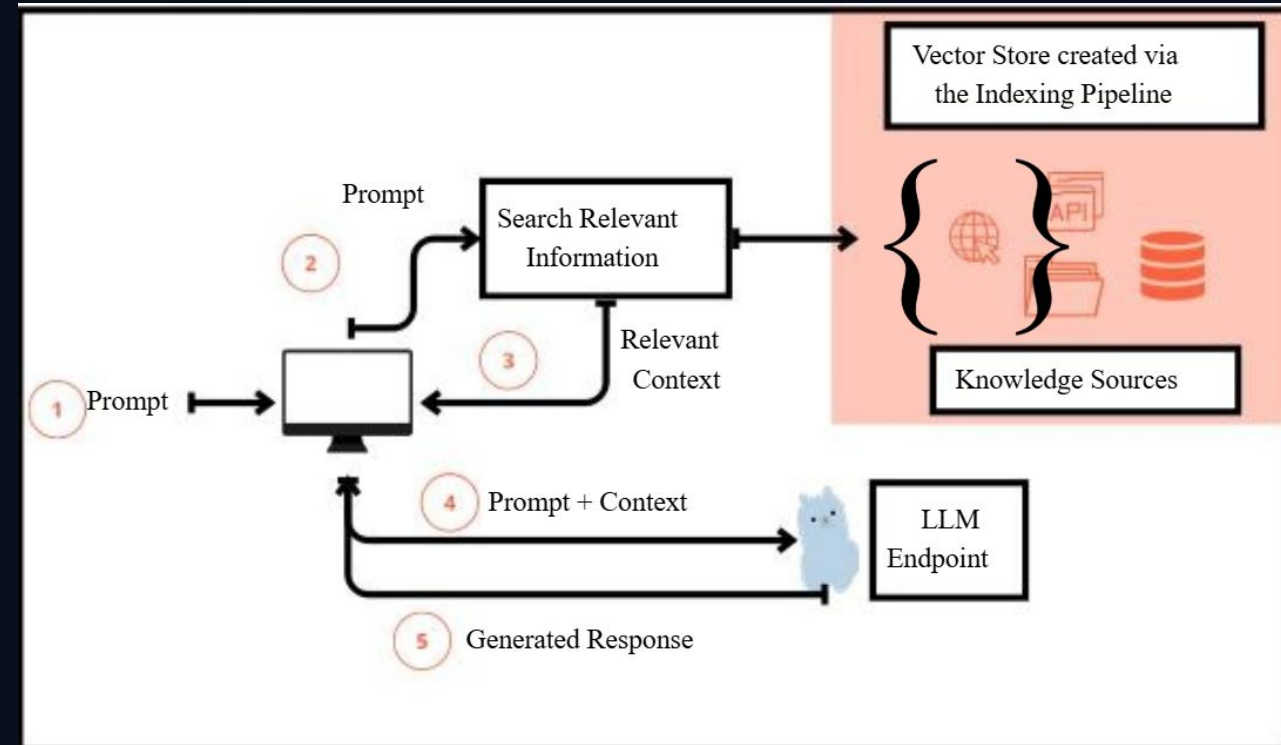


What is RAG?

Retrieval-Augmented Generation (RAG) is a framework that connects Large Language Models to dynamic, external data sources.

- **Knowledge Freshness:** Bypasses model training cutoffs which is very costly.
- **Reduced Hallucination:** Grounded in verifiable facts.
- **Cost Efficiency:** No retraining or fine-tuning required.

Model	Knowledge Cutoff	
GPT-3.5	September 2021	
GPT-4 (original)	April 2023	
GPT-4 Turbo	December 2023	
GPT-4.1	June 2024	
GPT-5	Varies by deployment; generally trained on data up to around mid-2024, but can use web search for newer information	



Phase 1: Data Ingestion



LangChain

Use cases: Good for applications that need enhanced AI capabilities, like language understanding tasks and more sophisticated text generation

Features: Stands out for its versatility and adaptability in building robust applications with LLMs

Agents: Makes creating agents using large language models simple through their agents API



LlamaIndex

Use cases: Good for tasks that require text search and retrieval, like information retrieval or content discovery

Features: Excels in data indexing and language model enhancement

Connectors: Provides connectors to access data from databases, external APIs, or other datasets

Collecting External Knowledge

The pipeline begins by fetching raw data from diverse sources including PDFs, or any data source provided..

- Text Extraction & Cleaning.
- Noise Removal (Headers/Footers).
- Metadata Enrichment (Source file, Page no).

we will be using LangChain

Phase 2: Chunking Strategies



Fixed-Size

Splitting text by character or token count (e.g., 512 tokens). Fast but can break sentences.



Semantic

Uses embeddings to find natural breaks where topics change. Higher retrieval accuracy.



Recursive

Attempts to split by headers, then paragraphs, then sentences to keep logic intact.

* Overlap (10-20%) is crucial to preserve context across chunk boundaries.

This is pretty old House. It needs to be demolished

raman

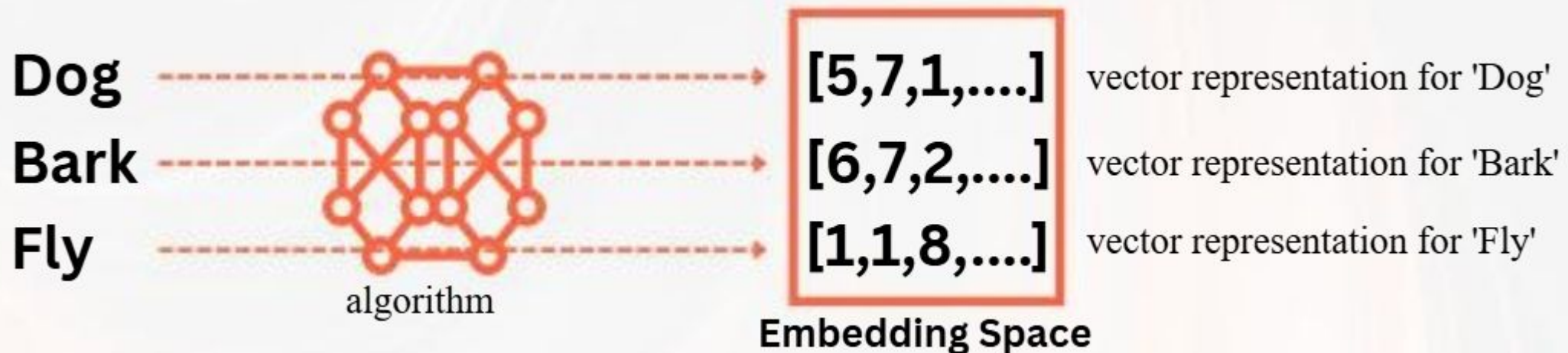
Phase 3: Embedding

Text chunks are transformed into dense, numerical vectors using transformer-based models.

→ Maps semantic meaning into 3D+ space.
in our project 384 D.

→ Similar meanings = Proximal vectors.

→ Enables mathematical search.

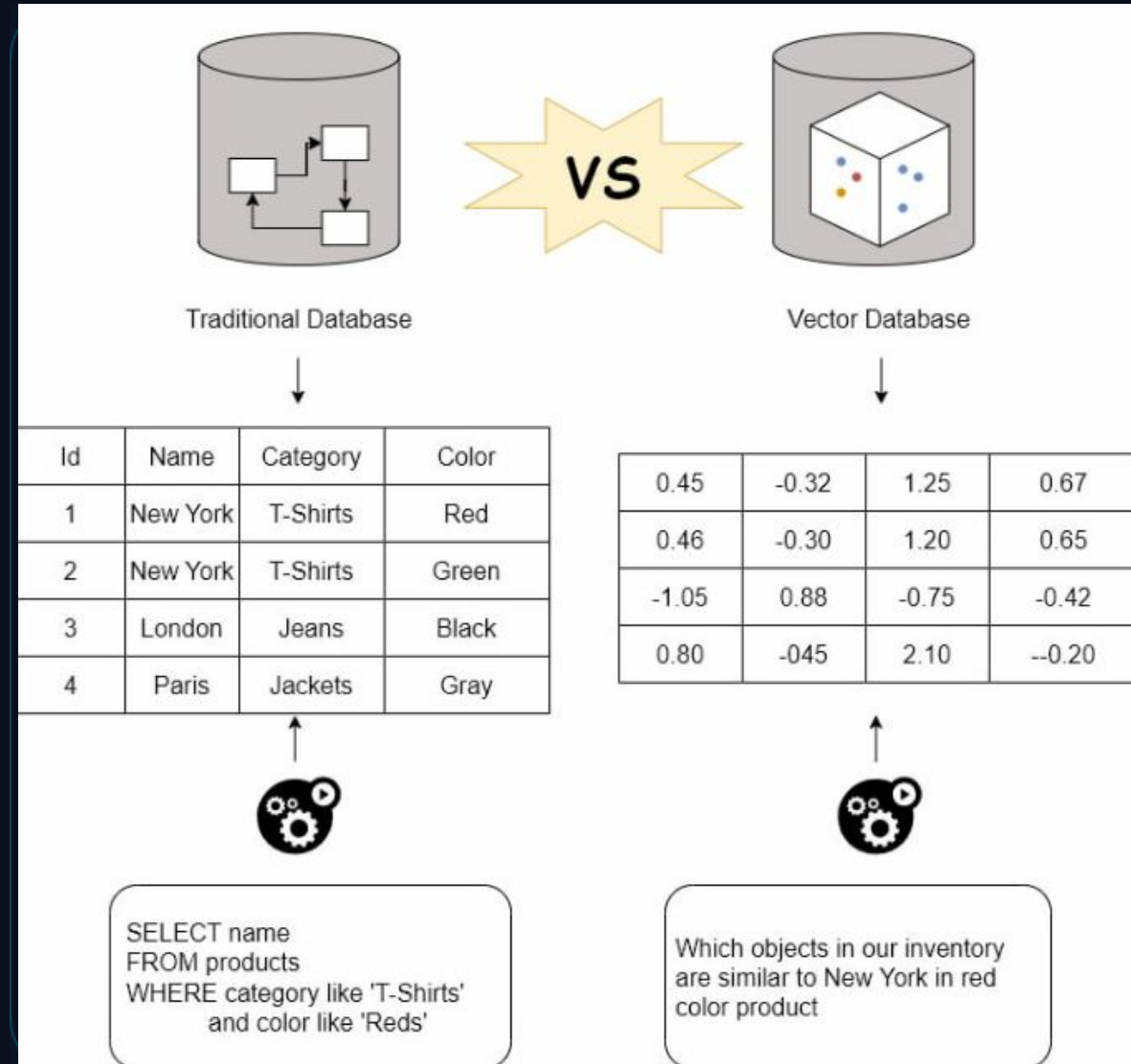


Phase 4: Vector Databases

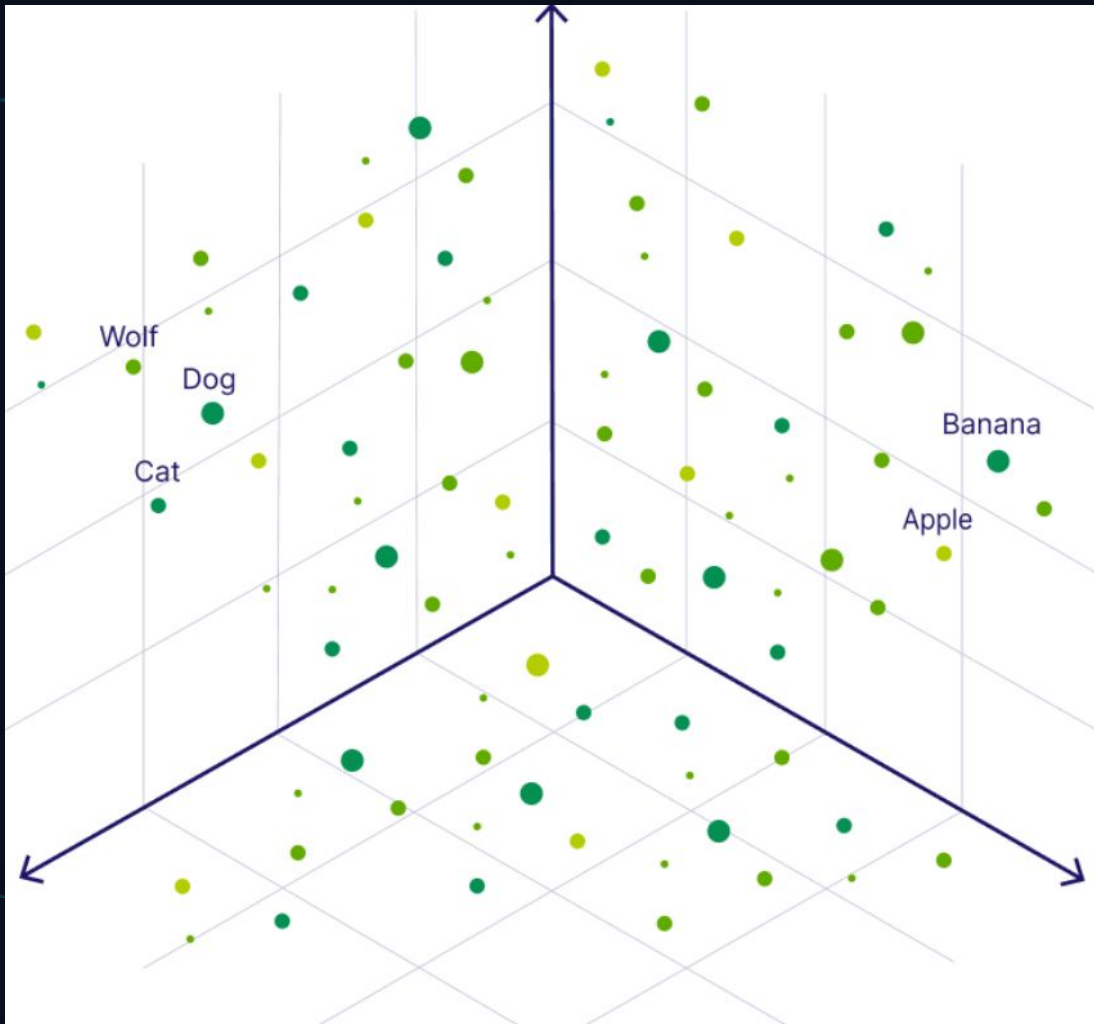
Optimized Storage

Unlike traditional SQL, vector DBs index high-dimensional data for similarity, not exact matches.

- **Metadata Filtering:** Combined keyword + vector search. Metadata includes info about source.
- **Examples:** Pinecone, Milvus, Weaviate ChromaDB



Retrieval: Query Embedding



Real-time Transformation

When a user submits a query, it undergoes the exact same embedding process as the stored data chunks.

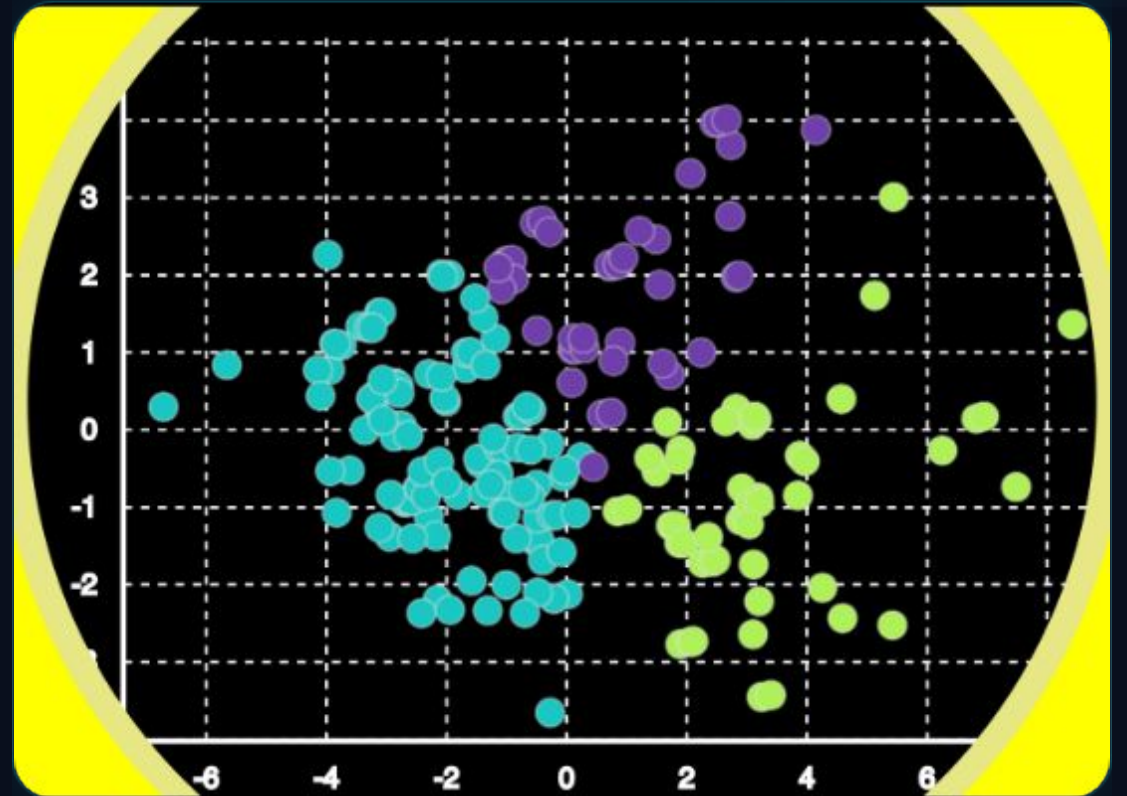
- Must have Same model, same dimensions.
- Converts natural language intent into a searchable coordinate and find top k matches.

Retrieval: Similarity Search

Finding the "Top-K"

The system performs a nearest-neighbor search to identify chunks most relevant to the query vector.

- **Cosine Similarity:** Measures angle between vectors.
- **L2 Distance:** Measures straight-line distance.
- **Re-ranking:** Fine-tuning the order of results for better context.



Prompt Engineering

```
### Simple RAG pipeline with Groq LLM
from langchain_groq import ChatGroq
import os
from dotenv import load_dotenv
load_dotenv()

### Initialize the Groq LLM (set your GROQ_API_KEY in environment)
groq_api_key = os.getenv("GROQ_API_KEY")

llm=ChatGroq(groq_api_key=groq_api_key,model_name="openai/gpt-oss-120b",temperature=0.1,max_tokens=1024)

## 2. Simple RAG function: retrieve context + generate response
def rag_simple(query,retriever,llm,top_k=3):
    ## retrieve the context
    results=retriever.retrieve(query,top_k=top_k)
    context="\n\n".join([doc['content'] for doc in results]) if results else ""
    if not context:
        return "No relevant context found to answer the question."

    ## generate the answer using GROQ LLM
    prompt=f"""Use the following context to answer the question concisely.
    Context:
    {context}

    Question: {query}

    Answer: """

    response=llm.invoke([prompt.format(context=context,query=query)])
    return response.content

answer=rag_simple("three reasons for forgetting",rag_retriever,llm)
print(answer)
```

Constructing the Context

The retrieved chunks are "stuffed" into a prompt template to provide the LLM with evidence.

"You are a helpful assistant. Use the following context to answer the user question:

[Retrieved Context Chunk 1,chunk 2,chunk 3,.....chunk k.]

Question: [User Query]"

The LLM Reasoning Engine

The Synthesis Brain

The Large Language Model doesn't just "copy-paste." It reasons over the provided context to synthesize a natural answer.

- Knowledge integration.
- Logic and reasoning application.
- Filtering irrelevant retrieved noise.

we will be using OpenAI/gpt-oss-120b

example code to call llm—>

```
from groq import Groq

client = Groq()
completion = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[
        {
            "role": "user",
            "content": ""
        }
    ],
    temperature=1,
    max_completion_tokens=8192,
    top_p=1,
    reasoning_effort="medium",
    stream=True,
    stop=None
)

for chunk in completion:
    print(chunk.choices[0].delta.content or "", end="")
```

Summary

The final response is generated, grounded in the retrieved sources.

- **Citations:** Ability to link back to source data.
- **Factuality:** Only answers based on context.

1. Loading our text file using **TextLoader,**
2. Splitting the text into chunks using **RecursiveCharacterTextSplitter,**
3. Creating embeddings using **all-MiniLM-L6-v2**
4. Storing the embeddings into **Chromadb**
5. Converting input query into a **vector embedding**
6. Retrieving chunks using **similarity_search_by_vector**

Output Generation



User
Query



Retrieved
Context



System
Instruction



Context
Augmented Prompt

Question

Who won the
2023 ICC
Cricket World
Cup?

Context

The 2023 Cricket
World Cup,
concluded on 19
November 2023,
with Australia
winning the
tournament.

Instruction

Answer the
question based
only on the
following context

Answer the question based
only on the following context :
*The 2023 Cricket World Cup,
concluded on 19 November
2023, with Australia winning
the tournament.*

Question - Who won the 2023
ICC Cricket World Cup?

QUESTIONS?

Retrieval-Augmented Generation: The Future of Domain-Specific AI

